How the PSO Search Runs

From a toy gravitational-wave test case to `global_optimizer_inspiral.py`

Student guide, companion to the PSO–κ project document

1 How to use this guide

Part A builds a particle swarm search from scratch on a small gravitational-wave problem, in about 30 lines of numpy, and shows every number and figure the real search produces. Part B takes the real code and maps each of its pieces onto Part A. Read A, run `pso_gw_toy.py`, then read B with the real code open beside it.

What you should be able to do afterwards

- Explain what the cost function is and why PSO minimises $-\max_t \rho(t)$.
- Say what each PSO setting (w, c_1, c_2, topology, swarm size, iterations) does.
- Explain why the spread of a converged swarm is *not* an error bar.
- Point to where each step of Part A lives in `global_optimizer_inspiral.py`.

Part A: A test case you can run in five minutes

2 The idea

A matched-filter search asks one question of the data $d(t)$ for every candidate waveform $h(\theta)$: how loud is the best match? With the noise-weighted inner product $(a|b) = 4\,\mathrm{Re}\int a^* b / S_n \, df$, the signal-to-noise ratio for a time shift t is

$$\rho(t;\theta) = \frac{|(d \,|\, h(\theta) e^{2\pi i f t})|}{\sqrt{(h|h)}}, \tag{1}$$

and one inverse FFT gives it for all t at once. The search statistic of a template is its best value, $\rho_{\max}(\theta) = \max_t \rho(t;\theta)$. A template bank evaluates this on a fixed grid of θ. PSO evaluates it at positions chosen on the fly.

2.1 The swarm

Each of n particles has a position x_i (a candidate θ) and a velocity v_i. Each iteration:

$$v_i \leftarrow w\, v_i + c_1 r_1 \,(p_i - x_i) + c_2 r_2 \,(g_i - x_i), \tag{2}$$

$$x_i \leftarrow x_i + v_i, \tag{3}$$

where p_i is the best place particle i has visited, g_i is the best place known to its neighbourhood, and $r_{1,2} \in [0, 1]$ are random numbers. The inertia w keeps a particle moving, c_1 pulls it back to its own best, and c_2 pulls it towards the group. In a *global-best* swarm every particle sees the whole swarm; in the *ring* topology of the real code (`k=5, p=2`) each particle sees only its 5 nearest neighbours, which slows convergence and helps exploration. The cost the swarm minimises is

$$C(\theta) = -\rho_{\max}(\theta). \tag{4}$$

3 The test case

Injected signal	$m_1 = 12$, $m_2 = 9\,M_\odot$ ($\mathcal{M}_c = 9.03\,M_\odot$, $\eta = 0.2449$), optimal SNR 20
Waveform	restricted TaylorF2, amplitude $f^{-7/6}$, phase to 1PN, cut at $f_{\rm ISCO}$
Noise	stationary Gaussian, coloured by an **analytic placeholder** aLIGO-like PSD
Band and data	20–512 Hz, 32 s at 1024 Hz, merger placed at $t = 20$ s
Search box	$\mathcal{M}_c \in [6, 14]\,M_\odot$, $\eta \in [0.10, 0.25]$
Swarm	25 particles, 100 iterations (2525 waveform evaluations)

The whole cost function is five lines:

```
def cost(x, data):
    h = template(x[0], x[1])              # 1. build the template at this
        position
    rho_t = snr_series(data, h)           # 2. matched filter, all time shifts,
         one FFT
    return -rho_t.max()                   # 3. loudest point in time; negate:
        PSO minimises
```

One physics detail matters: chirp masses must be converted to seconds with $GM_\odot/c^3 = 4.925 \times 10^{-6}$ s before they enter the phase. Without this conversion the phase barely depends on the mass and the whole problem becomes degenerate.

3.1 What the swarm is exploring

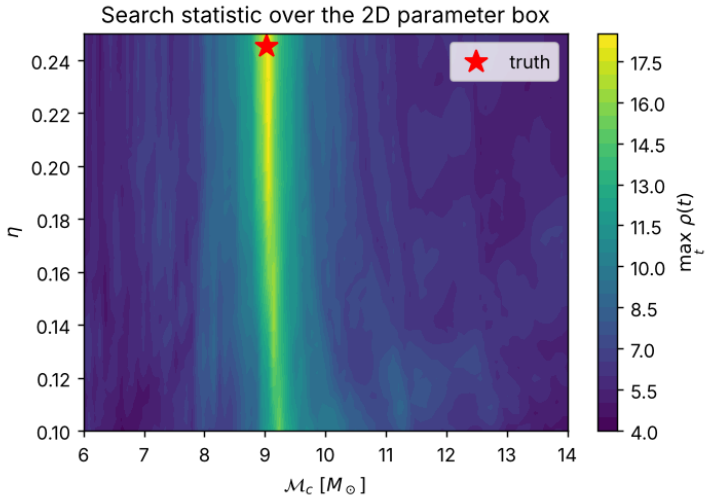


Figure 1: Search statistic $\rho_{\rm max}$ over the search box, computed on a 90×70 grid (6300 evaluations) for one noise realisation.

Two things stand out. The chirp mass is pinned sharply: the bright stripe is narrow in $\mathcal{M}_c$. The symmetric mass ratio is almost unconstrained along the stripe. This is the well-known $\mathcal{M}_c$–η structure of the inspiral phase, and it is the first reason a swarm does not return a unique answer to the second decimal. The largest grid value is $\rho = 18.46$, and the value at the true parameters is 18.94: with noise present the highest point need not be the true one.

3.2 Watching the swarm

The run ends at $\mathcal{M}_c^* = 9.025$, $\eta^* = 0.2498$ with $\rho = 19.00$, after 2525 evaluations. Note that η^* sits on the upper bound of the box while the true value is 0.2449. The optimum belongs to this

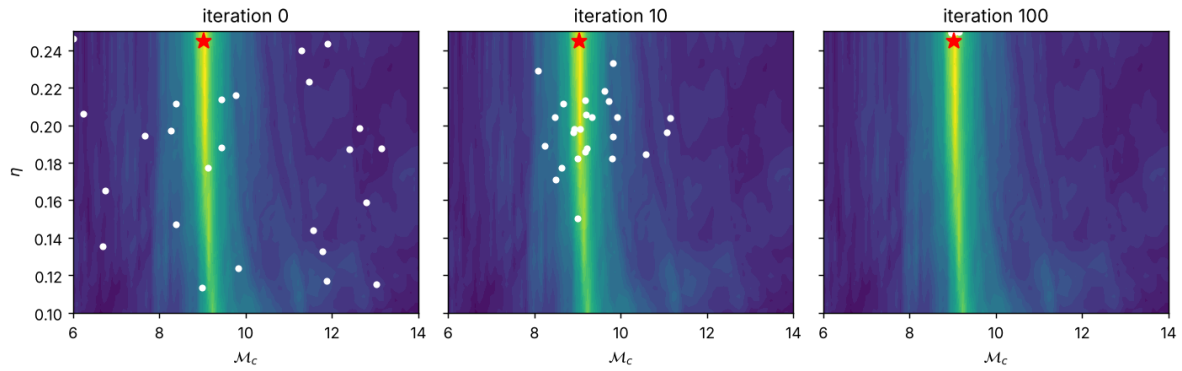


Figure 2: One run with the global-best swarm. Iteration 0 is random; by iteration 10 the particles have found the stripe; by iteration 100 they sit on top of each other.

noise realisation, not to the injection.

3.3 Settings matter

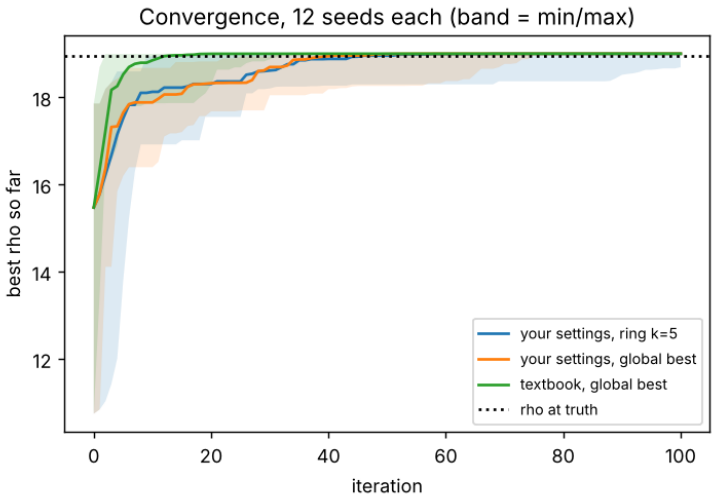


Figure 3: Best ρ so far versus iteration, 12 seeds per setting. Lines are medians, bands are min–max.

All three settings reach $\rho = 19.00$ in the median. They differ in speed: the textbook setting ($w = 0.7$, $c_1 = c_2 = 1.5$) is on the plateau after about 15 iterations, the setting in the real code ($w = 0.9$, $c_1 = c_2 = 0.4$) after about 45. With the ring topology one of the 12 runs ended at 18.69 after 100 iterations, so it had not finished. In this two-parameter problem the extra caution of the ring topology costs iterations and buys nothing; in the five-dimensional eccentric search the trade-off may differ, which is something to measure, not assume.

3.4 Why the swarm spread is not an error bar

The swarm width falls from 2.23 to $3 \times 10^{-4}\,M_\odot$, and 12 independent runs on the same data end at the same point. Both are properties of the optimiser, not of the measurement: a swarm collapses once it has found the top, however wide the top is. The measurement error is the scatter of the optimum over *different noise realisations*: Over 30 realisations at optimal SNR 20, the optimum is $\mathcal{M}_c^* = 9.030 \pm 0.013\,M_\odot$ (truth 9.028) and $\eta^* = 0.243 \pm 0.014$ (truth 0.245);

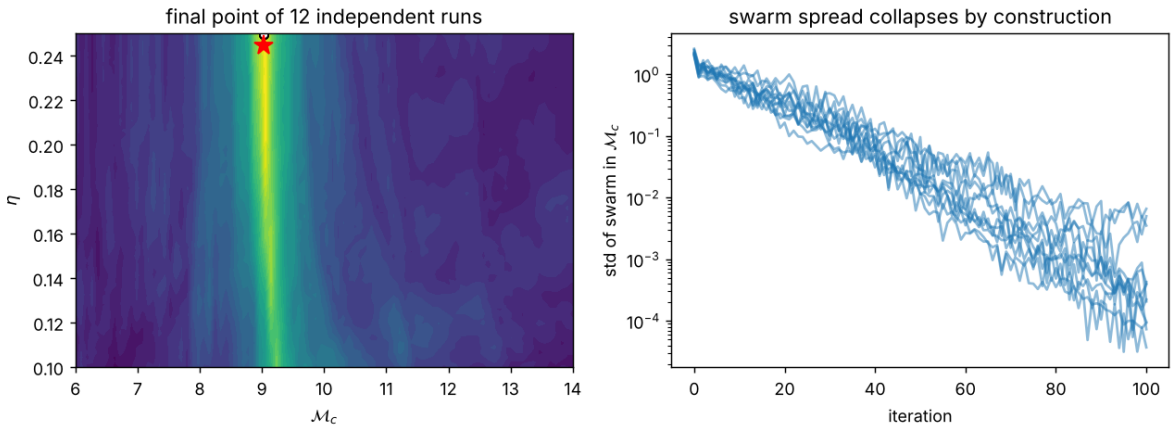


Figure 4: Left: final points of 12 independent runs on the same data, all on top of each other. Right: swarm spread in $\mathcal{M}_c$ during each run.

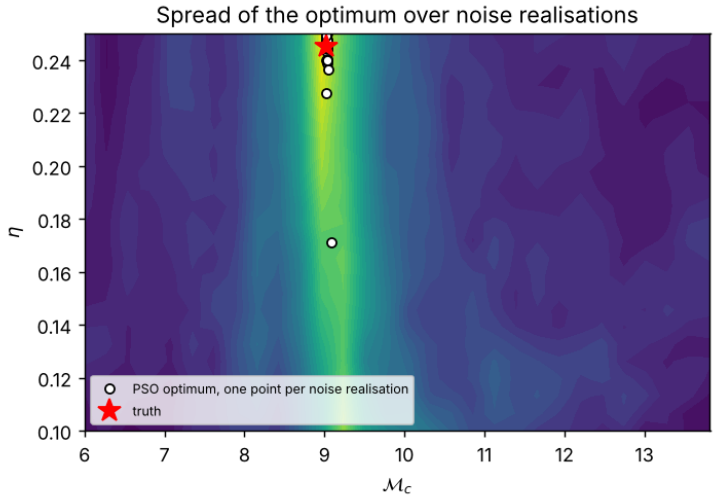


Figure 5: Optimum found by PSO for 30 different noise realisations of the same injection.

13% of runs end pinned at the upper bound $\eta = 0.25$. This scatter, about 0.15% in $\mathcal{M}_c$ and about 6% in η, is the statistical uncertainty of a point estimate. The swarm spread at the end of a single run (0.0003) is some 40 times smaller than the real scatter in $\mathcal{M}_c$ and tells you nothing about it. This is also why the project pairs PSO with a Fisher or Bayesian analysis for the uncertainty.

3.5 Signal versus noise only

Searching a whole parameter box raises the noise-only statistic well above zero: the median is 5.63 and the largest of 40 is 6.14. With an injection of optimal SNR 12 the median is 12.13 and the smallest is 9.73. This is the idea behind the foreground and background histograms of the real pipeline: a detection claim compares the best statistic in the data against the distribution of the best statistic in noise. With only 40 runs the noise tail is poorly known, which is why real background estimation needs far more data or time slides.

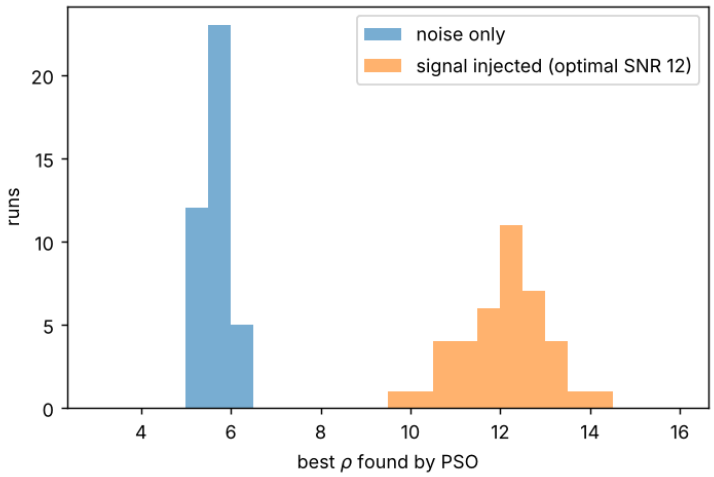


Figure 6: Best ρ found by PSO in 40 noise-only datasets and 40 datasets with an injected signal of optimal SNR 12.

3.6 Limits of the toy and exercises

The PSD is an analytic placeholder, the waveform is a 1PN restricted model, there is one detector, and the noise is Gaussian. None of these affects what the exercise teaches, but none of the numbers above are results about real detectors.

1. Replace `psd()` with the real aLIGO, ET and CE files (log-log interpolation) and compare the landscapes.
2. Lower the injected SNR to 8, 6, 5. At which SNR does the best noise-only value overtake the injected one?
3. Add χ_1 as a third search dimension. What happens to the stripe?
4. Add a toy eccentricity term to the phase and make e_0 the third dimension.
5. Set `ftol=1e-3, ftol_iter=10` and see how early the run stops, and what that does to the final spread.

4 From the toy to the real code: `global_optimizer_inspiral.py`

4.1 The same pipeline, one-to-one

Everything in the toy test case has a counterpart in the real search. The real code adds data handling, signal-consistency tests and a bookkeeping layer.

Toy test case	Real code	What is different
Analytic PSD `psd(f)`	`psd.associate_psds_to_segments(...)`; segments are divided by their PSD ("over-whitened")	PSD is estimated from the data (median Welch)
`make_data`	`strain.from_cli(opt)` reading a `.gwf` frame, high-passed, split into 512 s segments	Real noise is non-Gaussian and non-stationary
`template(Mc, eta)`	`FilterGenerator.generate_filter(p)` → `get_waveform_filter` with IMRPhenomD	IMR waveform; also stores a `template_hash` and duration
`snr_series(d, h)`	`MatchedFilterControl.matched_filter_and_cluster`	Adds time-clustering of triggers
$\rho_{\max} = \max_t \rho(t)$	`out_vals['stat'].max()`	The statistic is a *reweighted* SNR (see below), not raw ρ
`cost = -rho_max`	`sngl_stat_minimiser` returns `-stat.max()`	Same sign convention: PSO minimises
`pso(...)` (our 30 lines)	`pyswarms GlobalBestPSO` with `c1=c2=0.4, w=0.9, k=5, p=2`	Library handles boundaries and stopping
Best point θ^*	`run_pso` returns θ^*, cost, iterations, evaluations	—
Noise-only histogram (§3.5)	`global_coinc_findtrigs.py` foreground/background	Needs two detectors and a template-consistency cut

4.2 How the code runs, in order

1. **Configuration.** A hard-coded dictionary `opts` (frame file, channel, $f_{\text{low}} = 15$ Hz, `sample_rate` 2048, 512 s segments, `snr_threshold` 4, `approximant` IMRPhenomD). There is no `argparse` yet.

2. **Data.** The strain is read, high-passed at 15 Hz, cut into Fourier segments, and each segment is divided by its PSD. The χ^2 veto and a PSD-variation statistic are also prepared here.

3. **Search box.** Bounds are built from $m_1 \in [30, 50]$, $m_2 \in [20, 40]\, M_\odot$ and $\chi_{1z}, \chi_{2z} \in [-0.9, 0.9]$. They are converted to a rectangle in $(\mathcal{M}_c, \eta, \chi_{1z}, \chi_{2z})$, which is not the same region as the original (m_1, m_2) box.

4. **One PSO run per segment.** For each segment the same `optimizer` object calls `sngl_stat_minimiser` for every particle at every iteration (25 particles, up to 100 iterations $\approx$ 2500 waveforms).

5. **Inside the cost function** (the heart of the code):

   (a) convert $(\mathcal{M}_c, \eta, \chi_1, \chi_2) \to (m_1, m_2, \dots)$;

   (b) generate the frequency-domain filter;

   (c) matched-filter the whole 512 s segment and cluster the triggers;

   (d) compute the power χ^2, the PSD-variation value, and the ranking statistic `newsnr_sgveto_psdvar_threshold`;

   (e) **side effect:** register the template and *all* its clustered triggers in the event manager;

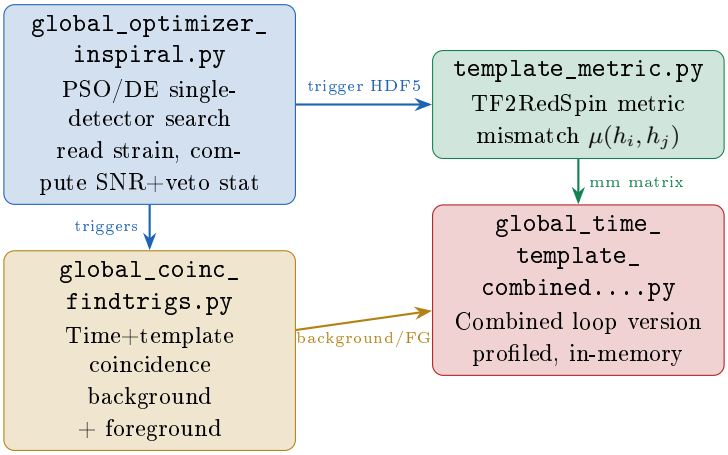
   (f) return $-\max(\text{stat})$ to the optimizer.

6. **Output.** `event_mgr.write_events` writes the trigger file; `write_template_info` writes a second file with the masses, spins and hash of every registered template. This second file exists because there is no template bank to look the templates up in.

Three consequences students should understand

1. **The cost is a maximum over the whole segment.** One PSO run looks for the single loudest event in a whole data segment (hundreds of seconds), not for an event at a known time. Weaker events are only seen through the side effect below.
2. **The trigger file is a by-product of the search path.** Every evaluated particle can leave triggers behind, so the file contains triggers from templates the swarm passed through, not only the best one. Triggers in the file are therefore not independent samples.
3. **The statistic is reweighted.** On real data a loud glitch has high raw SNR but a bad χ^2, and the reweighting pushes it down. In clean Gaussian simulations you want raw SNR, because fitting factors are defined on raw SNR.

4.3 The four files and what flows between them



Read the arrows as follows. The search is run once per detector (H1 and L1), giving a trigger file and a template-info file each. The coincidence step then asks, for every H1–L1 pair, whether the two templates are close in the TaylorF2 metric ($\mu < 0.03$) and whether the arrival times differ by less than 15 ms (foreground) or more than 100 ms (background). The two coincidence files do the same job; the combined one is a speed experiment and does not save foreground triggers.

4.4 Where the toy and the real search disagree: what to test first

Open items

Items marked **[verify]** come from reading the code, not from running it.

1. No injection has yet been recovered with this pipeline. The first job is the toy-style check on the real code: inject one known signal into Gaussian noise and require that $\mathcal{M}_c^*$ lands within a stated tolerance.
2. `sngl_stat_minimiser` calls `out_vals['stat'].max()`. If a template produces no trigger above `snr_threshold`, that array may be empty. **[verify]**
3. The same `optimizer` object is reused for every segment. Check whether the swarm is re-initialised each time. **[verify]**
4. PSO neighbourhoods (`k=5, p=2`) use distances in raw parameter units, where $\mathcal{M}_c$ (tens) dominates η (0.2–0.25). **[verify against the `pyswarms` documentation]**
5. `ftol=1e-10, ftol_iter=10` stops the run when the best cost stalls. This is why the final swarm spread says nothing about parameter uncertainty (Fig. 4).

6. The older scripts in `code_syed/` are Python 2.7, search only $(\mathcal{M}_c, \eta)$ in a $\pm$10–30% box around the injected chirp mass, and contain `m2_rec = args.mass1_rec`. Use them for reference only.